

Quixer: A Quantum Transformer Model

Nikhil Khatri Gabriel Matos Luuk Coopmans Stephen Clark

Quantinuum

17 Beaumont St., Oxford OX1 2NA, UK

{nikhil.khatri,gabriel.matos,luuk.coopmans,steve.clark}@quantinuum.com

June 7, 2024

Abstract

Progress in the realisation of reliable large-scale quantum computers has motivated research into the design of quantum machine learning models. We present Quixer: a novel quantum transformer model which utilises the Linear Combination of Unitaries and Quantum Singular Value Transform primitives as building blocks. Quixer operates by preparing a superposition of tokens and applying a trainable non-linear transformation to this mix. We present the first results for a quantum transformer model applied to a practical language modelling task, obtaining results competitive with an equivalent classical baseline. In addition, we include resource estimates for evaluating the model on quantum hardware, and provide an open-source implementation for classical simulation. We conclude by highlighting the generality of Quixer, showing that its parameterised components can be substituted with fixed structures to yield new classes of quantum transformers.

1 Introduction

Remarkable developments have been achieved in natural language processing, leading to the advent and popularisation of large language models (LLMs) [1–3]. At the same time, significant progress has been made in the field of quantum computing. While current quantum devices are still noisy [4], rapid improvements [5, 6] are driving the field into the error-corrected, fault-tolerant regime, where algorithms with asymptotic speed-up over their classical counterparts can be successfully run [7].

While powerful, LLMs are notoriously costly to train due to the number of parameters in state-of-the-art architectures [8]. Thus, it is of great practical interest to find alternative efficient, yet performant models. Given that quantum computers are known to provide a complexity-theoretic advantage in certain domains [9, 10], it is natural to explore what a quantum version of the transformer architecture could look like. While the original proposal for the Transformer model [11] uses a dot product self-attention mechanism, other architectures employ alternatives which are nonetheless performant e.g. the FNet [12].

In this work, we propose Quixer (QUantum mIXER), a quantum transformer model that incorporates quantum algorithmic primitives in a novel attention mechanism. A Linear Combination of Unitaries (LCU) [13] procedure is employed to create a superposition of token unitaries, and a Quantum Singular Value Transform (QSVT) [14] primitive is used to further apply a non-linear transformation to this superposition. Our primary contributions are the first result for a quantum transformer model applied to a practical language modelling task, along with a novel quantum attention mechanism built using the LCU and QSVT.

Figure 1 presents the high-level components of the Quixer model, which we describe in detail in Section 3. Section 2 provides the necessary background information on classical transformers and quantum computing. In Section 4, we present results for a language

modelling task on the Penn Treebank dataset, comparing the performance of Quixer against various contemporary classical models. Finally, in Section 5, we outline the extensible nature of Quixer, and highlight directions for future research.

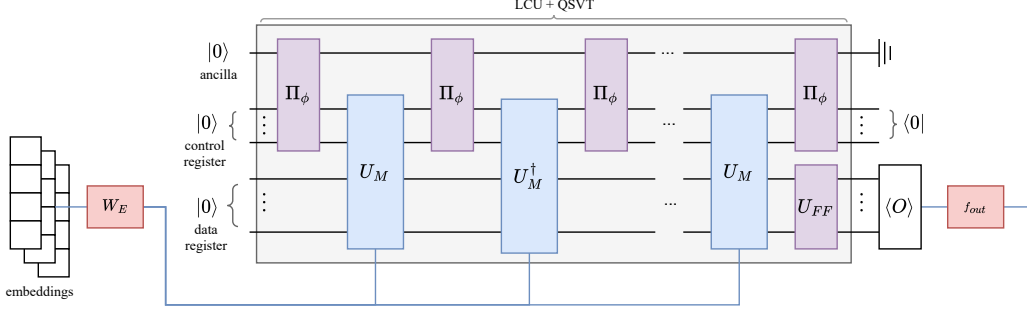


Figure 1: Quixer model architecture. Token embeddings are linearly mapped to angles by W_E . These angles parameterise unitary quantum circuits acting on a data register. A linear superposition of these unitaries is prepared using an LCU U_M , to which a polynomial, specified through the phases in the Π_ϕ gates, is applied through a QSVT procedure. A feed-forward unitary U_{FF} is then applied to the data register. Data is read out from the resulting quantum state using multiple measurement operators, and the resulting expectation values $\langle O \rangle$ are classically processed by f_{out} to produce the final output of the model.

1.1 Related work

Other types of quantum models have been proposed for natural language processing, such as quantum recurrent neural networks (QRNN) [15, 16]. In this section, we focus on literature concerning quantum models which are related either to transformers or language modelling. We refer the reader to Widdows et al. [16] for a more general overview of quantum natural language processing. To the best of our knowledge, other than the present work, the only previous results for language modelling using a quantum architecture have been presented by Basile and Tamburini [17], which uses a QRNN-like model to tackle a phone-recognition task on the TIMIT corpus.

Recently, there have been several proposals for quantum versions of the transformer architecture. Cherrat et al. [18] tackle a classification task using a family of MNIST datasets, and employ methods which differ significantly from ours. In particular, the authors use a data loading method to directly encode a trainable matrix on the amplitudes of a quantum state, and process this using a specialised “orthogonal layer” circuit. In contrast, Liao and Ferrie [19] and Guo et al. [20] attempt to directly quantise each component of the transformer architecture in a modular fashion. Their work is theoretical in nature and does not present results on a concrete language task.

Another proposal for a quantum transformer architecture is given by Zhao et al. [21], which directly incorporates a mechanism akin to Grover’s algorithm, and is trained on the Fashion MNIST dataset. Likewise, Gao et al. [22] use a Grover-like mechanism for sparse attention, though their work is fully theoretical. Finally, Zhao et al. [23] implement a transformer based on a quantum kernel method, and perform binary classification on MNIST and Fashion MNIST.

2 Background

2.1 The Transformer architecture

The heart of a Transformer is the **multi-head dot product self-attention mechanism** introduced by Vaswani et al. [11]. Several variations on the Transformer have been proposed in the literature, **which replace the dot product self-attention mechanism with alternate methods to mix information between tokens**. For instance, some substitute the quadratic-time dot product self-attention mechanisms with linear time attention mechanisms [24, 25]. Other variations exist which replace the attention unit with entirely untrainable transformations, such as the Fourier transform [12]. These variants demonstrate that the specific dot product self-attention mechanism is not necessary to make a performant transformer. Motivated by **the performance of these alternatives to the original transformer self-attention mechanism**, our focus in this work is not to quantise the dot product self-attention, but instead propose a novel form of token mixing built from quantum primitives.

2.2 Quantum computation

In circuit-based quantum computing, a number q of *qubits* is manipulated. The *state* $|\psi\rangle \in \mathbb{C}^2$ of each qubit is a normalised superposition (complex linear combination) of two *computational basis* states

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad |\alpha|^2 + |\beta|^2 = 1, \quad \alpha, \beta \in \mathbb{C}. \quad (1)$$

The joint state of the qubits is an element of the tensor product $\bigotimes_{j=1}^q \mathbb{C}^2$, which is isomorphic to $\mathbb{C}^{2^q}$ as a vector space; the computational basis of this space is $\{|c_{q-1}\dots c_0\rangle : c_k \in \{0, 1\}\}$. For a q -qubit state and a positive integer $j = \sum_{k=0}^{q-1} c_k 2^k$, $c_k \in \{0, 1\}$, we define $|j\rangle := |c_{q-1}\dots c_0\rangle$ (i.e. the k 'th qubit is set to $|c_k\rangle$; for instance, $|6\rangle = |110\rangle$). Quantum circuits are often drawn in diagrammatic form as in e.g. (5). In these diagrams, wires (represented by horizontal lines) can either be single qubits, or a group of qubits (i.e. a *register*) when crossed by an oblique line.

The state of a quantum system evolves through the application of *gates*, each of which is represented by a unitary matrix U (i.e. a matrix satisfying $U^\dagger U = I$, where $U^\dagger := \overline{U}^T$). Gates are drawn as boxes in circuit diagrams, and may act on single qubits or be *entangling*, i.e. act on multiple qubits. An example of a single-qubit gate is the R_X gate, defined as $R_X(\theta) := e^{-i\theta X}$, where X is a Pauli matrix [26]. The R_Y and R_Z gates are similarly defined in terms of the Y and Z Pauli matrices. The entangling gates we use are *controlled* gates of the form CU , where the action of a gate U on a *target* register depends on the state of a *control* register. For example, a CR_X gate applies a R_X gate to the target qubit when the control qubit is in state $|1\rangle$, and performs the identity operation I if the control is in state $|0\rangle$; if the qubit is in a superposition, the operation is applied to each basis element by linearity. Diagrammatically, a controlled gate is represented as a vertical line connecting a box on the target register to a circle on the control register, above which we indicate the state the gate is controlled on.

Measuring a qubit in state $|\psi\rangle$ with respect to the computational basis yields an output of 0 with probability $\|\langle 0|\psi\rangle\|^2$ and output 1 with probability $\|\langle 1|\psi\rangle\|^2$. Thus, quantum computation is inherently probabilistic; in general, several runs (often called *shots*) of a circuit are needed to obtain the desired result. For instance, some quantum algorithms rely on *postselection*, a process which discards all executions of a quantum circuit in which a postselected measurement did not yield a desired state. Postselecting in the $|0\rangle$ state is indicated in a circuit by a $\langle 0|$ placed near the end of a wire.

An *observable* is represented by a hermitian matrix O , i.e. a matrix satisfying $O^\dagger = \overline{O}^T = O$. The expectation value associated with an observable O applied to a state $|\psi\rangle$ is

given by

$$\langle O \rangle_{|\psi\rangle} := \langle \psi | O | \psi \rangle \in \mathbb{R} \quad (2)$$

Examples of common observables are the Pauli matrices introduced above. For further details on quantum computation, we refer the reader to [26].

3 Model

3.1 Unitary token embedding

As a first step, our model requires a quantum representation of each element of the vocabulary. Starting with a classical vector embedding $\vec{w}$ of a token w , we apply a linear layer W_E to obtain a set of angles $\vec{\theta}_w = W_E \vec{w}$. These are passed to a *parameterised quantum circuit* (PQC) U to prepare a unitary representation of w , $U_w := U(\vec{\theta}_w)$. PQCs are a common pattern in designing quantum machine learning models, and consist of parameterised gates, the angles of which are updated as part of a training procedure [27]. The specific choice of circuit U represents a trade-off between expressibility and circuit size, and has implications on the tractability of the gradient, an issue which we discuss in Section 6.

3.2 Mixing via Linear Combination of Unitaries

Now that we have prepared unitary circuit representations of the vocabulary, we implement the mixing of token information using the *linear combination of unitaries* (LCU) procedure [13]. Our aim is to have this mix take the form

$$M := \sum_{j=0}^{n-1} b_j U_j \quad (3)$$

for some (possibly trainable) complex parameters b_j satisfying $\sum_{j=0}^{n-1} |b_j| = 1$, where n is the window size. For this, we require a circuit U_{SEL} that applies a unitary U_k out of $\{U_j\}_{j=0, \dots, n-1}$ to a register conditional on the state of a control register being $|k\rangle$,

$$U_{\text{SEL}} = \sum_{j=0}^{n-1} |j\rangle\langle j| \otimes U_j, \quad U_{\text{SEL}}(|k\rangle \otimes I) = |k\rangle \otimes U_k. \quad (4)$$

A simple quantum circuit representation of U_{SEL} is shown below.

$$U_{\text{SEL}} = \begin{array}{c} \begin{array}{ccccccc} & |0\rangle & & |1\rangle & & \dots & & |n-1\rangle \\ \hline & \bullet & & \bullet & & & & \bullet \\ & | & & | & & & & | \\ U_0 & & U_1 & & \dots & & & U_{n-1} \end{array} \end{array} \quad (5)$$

When the control register is initially in some normalised state $|a\rangle = \sum_{i=0}^{n-1} a_i |i\rangle$, the above circuit can produce the weighted superposition of unitaries $\sum_{j=0}^{n-1} |a_j|^2 U_j$. However, this is only the case if the control register is postselected to remain in state $|a\rangle$ (we refer the reader to [28, 29] for more details on the LCU construction). This implies that, given a unitary U_{PREP} which prepares $|a\rangle$,

$$U_{\text{PREP}}|0\rangle = |a\rangle = \sum_{j=0}^{n-1} a_j |j\rangle, \quad (6)$$

the circuit

$$U_M = (U_{\text{PREP}}^\dagger \otimes I) U_{\text{SEL}} (U_{\text{PREP}} \otimes I), \quad (7)$$

diagrammatically represented as

$$U_M = \begin{array}{c} \text{---} \diagup \text{---} \\ \text{---} \end{array} \begin{array}{|c|} \hline U_{\text{PREP}} \\ \hline \end{array} \begin{array}{|c|} \hline U_{\text{SEL}} \\ \hline \end{array} \begin{array}{|c|} \hline U_{\text{PREP}}^\dagger \\ \hline \end{array} \begin{array}{c} \text{---} \\ \text{---} \end{array}, \quad (8)$$

prepares the superposition

$$(\langle 0| \otimes I)U_M(|0\rangle \otimes I) = M = \sum_{j=0}^{n-1} |a_j|^2 U_j \quad (9)$$

when the control register is prepared and postselected in the $|0\rangle$ basis state. We present the full derivation of Eq. (9) in Lemma 1 of Appendix A.2. Conjugating U_M with two projection operators yields M , as shown in Eq. (9). A matrix satisfying this criterion is said to be a *block encoding* of M . Note that the coefficients of the superposition on the right-hand side of the equation form an L1-normalised *real* vector. In our model, we prepare a superposition with complex coefficients, which can be achieved by adding a phase gate to the unitary associated with each token, such that

$$M := \sum_{j=0}^{n-1} |a_j|^2 U'_j = \sum_{j=0}^{n-1} e^{i\gamma_j} |a_j|^2 U_j. \quad (10)$$

Throughout the rest of the text, we assume the coefficients of the LCU to be $\{b_j\}_{j=0\dots n-1}$, where $b_j := e^{i\gamma_j} |a_j|^2 \in \mathbb{C}$.

3.3 Nonlinearity via Quantum Singular Value Transformation

The tools discussed thus far provide a method to prepare a linear combination of token embeddings encoded as unitary matrices. To provide richer interactions between the token unitaries, we use a method to prepare nonlinear transformations of this superposition. For this, we employ the **Quantum Singular Value Transform (QSVT) [14], which is able to apply polynomial transformations to a block-encoded matrix.** Given a block encoding U_M of a matrix M , and a real polynomial $P_{\tilde{c}}$ of degree d , defined as:

$$P_{\tilde{c}}(x) = c_d \cdot x^d + c_{d-1} \cdot x^{d-1} + \dots + c_1 \cdot x + c_0 \quad (11)$$

with the condition that $P_{\tilde{c}}$ obeys

$$|P_{\tilde{c}}(x)| \leq 1, \quad \forall x \in [-1, 1] \quad (12)$$

$$\text{parity}(P_{\tilde{c}}) = d \bmod 2 \quad (13)$$

the QSVT provides a circuit which prepares the matrix:

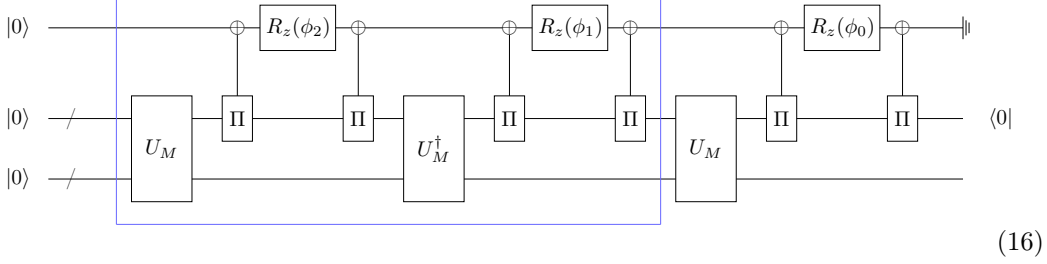
$$P_{\tilde{c}}(M) = c_d M^d + c_{d-1} M^{d-1} + \dots + c_1 M + c_0 I \quad (14)$$

The full expression for the QSVT is

$$\begin{cases} \Pi_{\phi_1} U_M \left[\prod_{k=1}^{\frac{d-1}{2}} \Pi_{\phi_{2k}} U_M^\dagger \Pi_{\phi_{2k+1}} U_M \right], & d \text{ odd}, \\ \left[\prod_{k=1}^{\frac{d}{2}} \Pi_{\phi_{2k-1}} U_M^\dagger \Pi_{\phi_{2k}} U_M \right], & d \text{ even}, \end{cases} \quad (15)$$

where $\Pi_\phi := e^{i\phi(2|0\rangle\langle 0| - I)}$ acts as a controlled R_Z rotation on a newly introduced ancilla qubit, and the angles ϕ_k are fixed and chosen so as to implement the desired polynomial (see [14, 28]). A circuit implementing the QSVT for a cubic polynomial (i.e. $d = 3$) is illustrated below, where $\oplus$ is a NOT operation that is applied if the control register is in

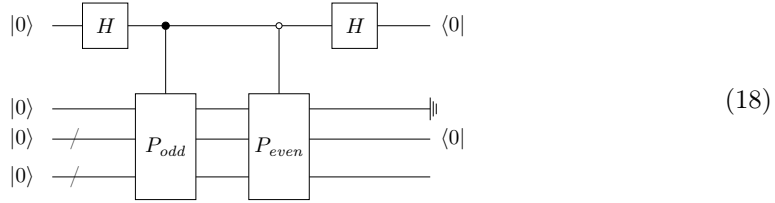
state $|0\rangle$; this condition is represented by the projection $\Pi := |0\rangle\langle 0|$. The blue box highlights the subcircuit which is repeated for higher degree polynomials.



We would like to be able to effect polynomial transformations of M which do not have parity $\in \{0, 1\}$. For this, we observe that any polynomial can be split into the sum of an even (all even powers) and an odd (all odd powers) parity polynomial

$$P_{\vec{c}} = P_{\text{odd}} + P_{\text{even}}. \quad (17)$$

Using this decomposition, it is possible to prepare $P_{\vec{c}}(M)$ with unknown parity using the following LCU circuit, using one additional postselected ancilla qubit.



3.4 Quixer

Having described each of the components of Quixer, we now provide an end-to-end description of the model. Given an input sequence $\{w_j\}_{j \in 0 \dots n-1}$, Quixer begins by preparing a q -qubit unitary circuit $U_{w_j} := U(\theta_{\vec{w}_j})$ for each token, using a trainable linear matrix W_E to obtain a set of PQC for each token embedding $\theta_{\vec{w}} = W_E \vec{w}$, as described in Section 3.1.

An LCU circuit U_M is then instantiated with trainable complex coefficients $\{b_j\}_{j \in 0 \dots n-1}$, to prepare a linear combination of the token unitaries $M_{\vec{b}, \theta} = \sum_j^{n-1} b_j U_{w_j}$ as described in Section 3.2. Next, a polynomial $P_{\vec{c}}$ of degree d with trainable coefficients $\vec{c}$ is applied to the LCU via a QSVT to obtain $P_{\vec{c}}(M_{\theta})$, as outlined in Section 3.3. This circuit is then applied to a $|0\rangle$ state on the data register, followed by a trainable PQC U_{FF} on the data register, resulting in the final (unnormalised) quantum state

$$|\psi\rangle = U_{FF} P_{\vec{c}}(M_{\vec{b}, \theta}) |0\rangle. \quad (19)$$

Information from this state is then read out by measuring multiple expectation values

$$o_k = \langle O_k \rangle_{|\psi\rangle}, \quad (20)$$

resulting in a classical real vector $\vec{o}$. The final output of the model is then obtained by applying a fully-connected feed-forward neural network f_{out} to $\vec{o}$,

$$\vec{y} = f_{out}(\vec{o}). \quad (21)$$

3.4.1 Attention in Quixer

In the classical transformer, the dot product self-attention mechanism operates by preparing a sum of *value* vectors, weighted by pairwise interactions captured using *query-key* products [11].

In contrast, Quixer captures interactions between multiple tokens through the composition of their representative unitaries. A weighted sum of such interactions is computed to prepare the final state. While sequential composition with a unitary preserves the norm of the quantum state, it changes the magnitude of the expectation value for a particular observable. This allows Quixer to model attention through unitary composition. For instance, when implementing a quadratic polynomial using the QSVT procedure, Quixer prepares the quantum state described by

$$P_{\vec{c}}(M_{\vec{b},\theta})|0\rangle = c_2 \sum_{j,k=0}^{n-1} b_j b_k U_j U_k |0\rangle + c_1 \sum_{j=0}^{n-1} b_j U_j |0\rangle + c_0 |0\rangle. \quad (22)$$

The action of this model can be seen as the sum of pairwise interactions (again, captured through composition of word unitaries), single-token terms, and a final bias term. Here pairwise interactions are computed between all *skip-bigrams* (pairs of tokens, not necessarily adjacent) in the context. Higher degree polynomials, analogously, compute interactions between skip- k -grams; we write this out in Appendix A.1.

3.5 Resource estimates

3.5.1 Postselection probability

As discussed in Section 3.2, the preparation of the LCU $M_{\vec{b},\theta}$ is contingent on postselecting the control register in state $|0\rangle$. The probability of this succeeding, and of producing the desired state $M_{\vec{b},\theta}|0\rangle$, is equal to

$$p_M = \|M_{\vec{b},\theta}|0\rangle\|^2 \quad (23)$$

$$= \langle 0 | M_{\vec{b},\theta}^\dagger M_{\vec{b},\theta} | 0 \rangle \quad (24)$$

$$= \sum_{j,k=0}^{n-1} \bar{b}_j b_k \langle 0 | U_j^\dagger U_k | 0 \rangle \quad (25)$$

$$= \sum_{j < k} \bar{b}_j b_k \langle 0 | U_j^\dagger U_k | 0 \rangle + \bar{b}_k b_j \langle 0 | U_k^\dagger U_j | 0 \rangle + \sum_{j=0}^{n-1} |b_j|^2 \quad (26)$$

$$= \sum_{j < k} 2 \cdot \text{Re}[\bar{b}_j b_k \langle 0 | U_j^\dagger U_k | 0 \rangle] + \sum_{j=0}^{n-1} |b_j|^2. \quad (27)$$

In turn, the probability of successfully preparing the desired polynomial using the QSVT is

$$p = \|P(M_{\vec{b},\theta})|0\rangle\|^2. \quad (28)$$

However, neither expression yields a lower bound on the success probability, since the minimum value of both is 0 without further assumptions on the circuit structure. Previous works have lower bounded the success probability of LCU and QSVT circuits through spectral analysis of the matrices M and $P(M)$ [14, 30]. Such analysis, however, requires additional assumptions on the training algorithm, parameter space and unitary token embedding choice, as it depends on the concrete form that these unitaries take. Thus, in general, our model is not immediately amenable to such analysis without implementing further constraints. For the concrete Quixer implementation provided in Section 4, we evaluate the postselection probability empirically.

3.5.2 Gate and qubit complexity

Given a sequence of length n , our model uses q qubits for the data register, $\lceil \log_2(n) \rceil$ qubits for the control register, and 3 ancillae: one for the QSVT procedure, one for the combination

of odd and even parity terms, and one for the implementation of the QSVT projectors Π (detailed below, and in Appendix A.3). Thus, the asymptotic total number of qubits is

$$O(q + \log_2(n)). \quad (29)$$

The runtime of a quantum circuit is characterised by its gate complexity. Following the construction in Appendix A.3, a single-qubit gate can be controlled on a given number of qubits using a gate count linear in that number. Let us assume that each token unitary has at most g gates when written in terms of single-qubit operations controlled on at most one qubit (note that g is well-defined, since any unitary can be decomposed into CX gates and single-qubit gates [26]). Then, each token unitary controlled on $\log_2(n)$ qubits can be implemented using $O(g \log_2(n))$ gates, implying that the LCU component of the circuit can be implemented using $O(n g \log_2(n))$ gates. As explained in Appendix A.3, each projection Π in the QSVT can also be implemented using a number of gates linear in the number of qubits in the control register. For a QSVT implementing a polynomial of degree d , this yields an overall asymptotic gate count of $O(d n g \cdot \log_2(n))$. If each unitary is a PQC with l layers, each of which contains a number of parameterised gates proportional to the qubits in the data register (i.e. $g \propto ql$), this corresponds to $O(d n q l \cdot \log_2(n))$.

The above complexity can be further improved by leveraging the technique in [31]. For an additional $\log_2(n) - 2$ ancilla qubits (which does not change the qubit complexity in Eq. (29)), each token unitary controlled on $\log_2(n)$ qubits can be implemented using $O(g)$ instead of $O(g \log_2(n))$ gates. This results in a gate complexity of

$$O(d n g), \quad (30)$$

or, again, if each unitary is a PQC with l layers, each of which contains a number of parameterised gates proportional to the qubits in the data register (i.e. $g \propto ql$),

$$O(d n q l). \quad (31)$$

4 Experimental results

4.1 Setup

To evaluate our model in a practical setting, we apply an instance of Quixer to a language modelling task. Here, given a sequence of words $\{w_j\}_{j=0\dots n-1}$ (which we take to be our tokens), the model must predict the subsequent word w_n . We evaluate our model on the Penn Treebank (PTB) dataset, which consists of 966K training tokens, 77K validation tokens, and 86K test tokens [32]. We obtain this from the HuggingFace datasets package¹.

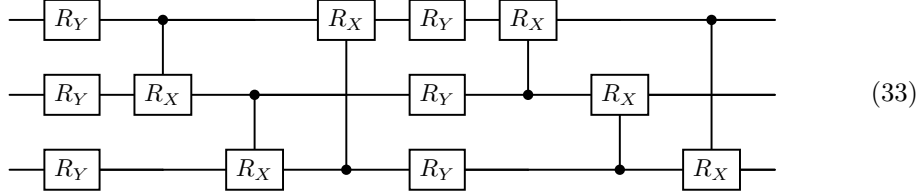
We implemented Quixer as a Torch module, and used TorchQuantum [33], a Torch-native quantum computation framework, to simulate the PQCs and compute expectation values. When simulating the model classically, it is not necessary to prepare the explicit circuits for the LCU and QSVT operations. Instead, we apply each token unitary to a copy of the data register, and directly prepare a linear combination weighted by $\vec{b}$. The polynomial implemented by the QSVT can then be directly computed by repeating the LCU application on the data register d times. For further details on the practical classical simulation of Quixer, we refer the reader to the code provided in the GitHub repository github.com/CQCL/Quixer.

Each token unitary is composed of 4 layers of “circuit 14” out of several parameterised circuits that Sim et al. [34] studies. Our choice is motivated by the high expressibility and entangling capability of this circuit. Note, however, that this choice is not canonical, and any parameterised circuit can be employed in our architecture. One layer of this circuit alternates layers of R_Y and CR_X gates as follows

$$\prod_{j=1}^q CR_{X_{j,j+1}}(\theta_{4,j}) \prod_{j=1}^q R_{Y_j}(\theta_{3,j}) \prod_{j=1}^q CR_{X_{j,j-1}}(\theta_{2,j}) \prod_{j=1}^q R_{Y_j}(\theta_{1,j}). \quad (32)$$

¹https://huggingface.co/datasets/ptb_text_only

For l layers of circuit 14 acting on q qubits, the number of parameterised gates (and parameters) is $4lq$. Below, we show a single layer of this circuit on 3 qubits.



We implement a trainable cubic polynomial. For this configuration, each token unitary has 96 parameters, which we prepare by applying W_E to a 512-dimensional word embedding. In our experiments, we compute the expectation values of the X , Y and Z Pauli operators independently for each qubit, resulting in a vector $\vec{\sigma} \in \mathbb{R}^{3q}$. f_{out} , consisting of a 2-layer feed-forward neural network with a ReLu nonlinearity in between, is used to map the expectation values to a probability distribution over tokens.

We compare Quixer against an LSTM [35] and the Transformer [11], as provided in the PyTorch [36] package, and a PyTorch implementation of the FNet [12], which we include in this project’s source code. The LSTM represents a high-performant, non-attention-based architecture. FNet is a simplified version of the Transformer architecture which replaces the multi-head self-attention unit with a 2-dimensional Fourier transform applied to the input matrix. The residuals, layer normalisation and MLPs from the original Transformer model are retained. Finally, the Transformer is the main component of the majority of modern language models.

Quixer processes a sequence of tokens of length n and produces a single subsequent token as output. We use a context length of 32, and stride by 1 token per step to generate each token in the dataset. We provide the classical baselines with identical setup of window size and stride. The performance of each model is measured by *perplexity* (PPL), which is defined as the exponent of the cross entropy loss; see [37] for more details. For each of the classical baselines, we use embedding sizes of 96 and 128. An embedding dimension of 96 restricts the classical models to the number of angles Quixer uses to parameterise each token unitary.

All models were trained using the Adam optimiser [38], and the learning rate was varied according to a cosine annealing schedule [39]. Each batch contains 32 contexts, and each context produces 32 tokens, yielding an effective batch size of 1024. All models were trained for 30 epochs, and the best epoch was selected based on perplexity on the validation set. Learning rates, dropout, and weight decay were tuned per model, and are described in Appendix A.4. Quixer was trained on one A100 GPU, on which 30 epochs took 3 hours 45 minutes.

4.2 Results

Table 1 shows the perplexities obtained by each model for the word-level language modelling task on the PTB dataset. All results are reported averaged over 10 runs, along with error bars of one standard deviation. We observe that Quixer outperforms both sizes of LSTM, performs competitively with the 96-dimension FNet, and only marginally worse than the 128-dimension FNet. The Transformer outperforms all other models, even with a 96-dimension embedding.

We obtain results similar to FNet, a model which is known to achieve results competitive with Transformer-based language models at scale [12]. This is an encouraging result for a first quantum transformer applied to language modelling, and provides a validation of the Quixer architecture. Note, however, that these results are far from those obtained by state-of-the-art models, and that a direct comparison with such models is not the objective of this work.

Figure 2 plots the distribution of postselection probabilities on the PTB test dataset for 10 runs of our model with the same hyperparameters, but different seeds. The minimum

Model	Dimension	Layers	PPL
LSTM	96	2	144.3 (± 9.1)
	128	2	127.1 (± 3.1)
FNet	96	2	120.5 (± 1.0)
	128	2	117.7 (± 0.8)
Transformer	96	1	100.1 (± 0.2)
	128	1	97.0 (± 0.3)
Quixer (Ours)	6 qubits	cubic	122.0 (± 2.2)

Table 1: Results for word-level language modelling on Penn Treebank

of mean success probabilities across a run is 0.0159. The mean success probability across all runs is 0.0757 (± 0.0460). As a reference, the Boltzmann machine, a model which can be recovered as a special case of Quixer (see Section 5), involves the preparation of Gibbs states [14, 40]. This has a success probability lower bound given by 2^{-q} , which evaluates to 0.0156 for $q = 6$. Further analysis is necessary to characterise the scaling of our model’s success probability with system size.

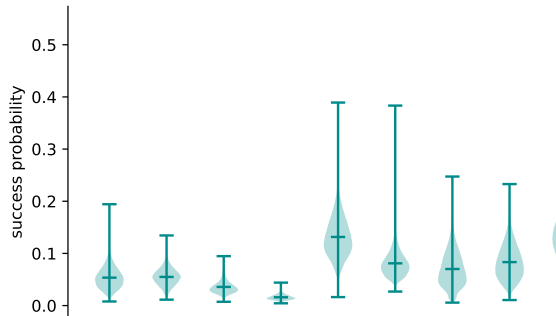


Figure 2: Distribution of postselection success probabilities (as in Eq. (28)) across 10 different seeds. Horizontal bars indicate mean and extrema for each seed.

5 Quixer as a framework

Our experiments employ a very flexible instance of Quixer, with several sets of parameters trained in-task. Subsets of these, however, may be fixed *a priori* to yield new models. The polynomial coefficients $\vec{c}$ need not be trained in-task, and specific polynomials may be chosen to achieve a desirable balance between expressibility, success probabilities, and magnitudes of the gradients. For example, fixing the polynomial to be an approximation of the matrix exponential and the token unitaries to be single Pauli operators, one recovers the quantum Boltzmann machine [40].

The instance of Quixer presented in Section 4 is a single layer implementing a cubic nonlinearity. By repeating the QSVT circuit for a specific polynomial $P_{\vec{c}}$, with different LCU encodings of the data, it is possible to meaningfully extend Quixer to a multi-layer setting. For instance, a 2-layer Quixer model implemented in this manner would prepare the state

$$|\psi\rangle = U_{FF}P_{\vec{c}}(M_{\vec{b},\theta'})P_{\vec{c}}(M_{\vec{b},\theta})|0\rangle. \quad (34)$$

Note also that the token unitaries U_w in our experiments were chosen to be highly-expressive PQCs. This is known to hamper the trainability of the model as the number

of qubits increases [41]. This expressivity may be adjusted to mitigate this problem, for example through the use of matchgate circuits [42]. We discuss this issue further in Section 6.

6 Limitations

A significant challenge faced by contemporary quantum machine learning models is that the currently available methods to obtain gradients on a quantum computer have been shown to take time polynomial in the number of parameters [43], which is prohibitive if this number is to approach those used in modern large language models. Another well-known problem faced by quantum machine learning models is a concentration of measure phenomenon that causes gradients to exponentially vanish as the number of qubits in the model increases [41]. While some quantum models are not affected by this, such as those comprised of matchgate circuits [42], it is believed that most models evading this issue can be simulated classically [44], precluding any quantum advantage. Finding an instance of Quixer that does not suffer from vanishing gradients while being expressive enough to not be amenable to classical simulation is left to future work.

As outlined in Section 4, training and evaluation has been done fully classically in this work. While this is representative of the model’s performance, it is necessary to consider implementation overheads on a real device, along with any noise if running on an architecture which is not fully fault tolerant. Moreover, comparisons between classical models and quantum counterparts can be difficult due to their inherently different data representations. While typical classical models manipulate parameters and intermediate representations as real vectors, quantum models are limited to low-dimensional parameterisation of unitary matrices on exponential-sized complex-valued systems. Another limitation of the results presented here is the relatively small scale of models considered, as the exponential scaling of vector space dimension with the number of qubits precludes the simulation of large-scale quantum systems.

7 Conclusion

In this work, we have described Quixer, a new quantum transformer architecture. We successfully applied it to a language modelling task on the Penn Treebank dataset, obtaining encouraging results. This represents the first example of a quantum transformer applied to a real-world language modelling task. We further described how Quixer can be extended to a family of quantum transformer models by e.g. making a particular choice of the coefficients of the polynomial transformation effected by the QSVT. Future work will involve finding new instances of this framework which make favourable trade-offs between gradient magnitudes and classical simulability, to scale the model up closer to the performance of contemporary classical language models.

8 Acknowledgements

We thank Tuomas Laakkonen, Frédéric Sauvage, Marcello Benedetti and Konstantinos Meichanetzidis for feedback on this manuscript. We further thank Tuomas Laakkonen for insightful discussions, for suggesting the use of the Toffoli construction in [45], and for suggesting the use of the technique in [31] to reduce the complexity of our model in Section 3.5.2.

References

- [1] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez,

- Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023.
- [2] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, Pouya Tafti, Léonard Hussenot, Pier Giuseppe Sessa, Aakanksha Chowdhery, Adam Roberts, Aditya Barua, Alex Botev, Alex Castro-Ros, Ambrose Slone, Amélie Héliou, Andrea Tacchetti, Anna Bulanova, Antonia Paterson, Beth Tsai, Bobak Shahriari, Charline Le Lan, Christopher A. Choquette-Choo, Clément Crepy, Daniel Cer, Daphne Ippolito, David Reid, Elena Buchatskaya, Eric Ni, Eric Noland, Geng Yan, George Tucker, George-Christian Muraru, Grigory Rozhdestvenskiy, Henryk Michalewski, Ian Tenney, Ivan Grishchenko, Jacob Austin, James Keeling, Jane Labanowski, Jean-Baptiste Lespiau, Jeff Stanway, Jenny Brennan, Jeremy Chen, Johan Ferret, Justin Chiu, Justin Mao-Jones, Katherine Lee, Kathy Yu, Katie Millican, Lars Lowe Sjoesund, Lisa Lee, Lucas Dixon, Machel Reid, Maciej Mikula, Mateo Wirth, Michael Sharman, Nikolai Chinaev, Nithum Thain, Olivier Bachem, Oscar Chang, Oscar Wahltinez, Paige Bailey, Paul Michel, Petko Yotov, Rahma Chaabouni, Ramona Comanescu, Reena Jana, Rohan Anil, Ross McIlroy, Ruibo Liu, Ryan Mullins, Samuel L Smith, Sebastian Borgeaud, Sertan Girgin, Sholto Douglas, Shree Pandya, Siamak Shakeri, Soham De, Ted Klimenko, Tom Hennigan, Vlad Feinberg, Wojciech Stokowiec, Yu hui Chen, Zafarali Ahmed, Zhitao Gong, Tris Warkentin, Ludovic Peran, Minh Giang, Clément Farabet, Oriol Vinyals, Jeff Dean, Koray Kavukcuoglu, Demis Hassabis, Zoubin Ghahramani, Douglas Eck, Joelle Barral, Fernando Pereira, Eli Collins, Armand Joulin, Noah Fiedel, Evan Senter, Alek Andreev, and Kathleen Kenealy. Gemma: Open Models Based on Gemini Research and Technology, 2024.
 - [3] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b, 2023.
 - [4] John Preskill. Quantum Computing in the NISQ era and beyond. *Quantum*, 2:79, August 2018. ISSN 2521-327X. doi: 10.22331/q-2018-08-06-79. URL <http://dx.doi.org/10.22331/q-2018-08-06-79>.
 - [5] M. P. da Silva, C. Ryan-Anderson, J. M. Bello-Rivas, A. Chernoguzov, J. M. Dreiling, C. Foltz, F. Frachon, J. P. Gaebler, T. M. Gatterman, L. Grans-Samuelsson, D. Hayes, N. Hewitt, J. Johansen, D. Lucchetti, M. Mills, S. A. Moses, B. Neyenhuis, A. Paz, J. Pino, P. Siegfried, J. Strabley, A. Sundaram, D. Tom, S. J. Wernli, M. Zanner, R. P. Stutz, and K. M. Svore. Demonstration of logical qubits and repeated error correction with better-than-physical error rates, 2024.
 - [6] Robert D. Delaney, Lucas R. Sletten, Matthew J. Cich, Brian Estey, Maya Fabrikant, David Hayes, Ian M. Hoffman, James Hostetter, Christopher Langer, Steven A. Moses, Abigail R. Perry, Timothy A. Peterson, Andrew Schaffer, Curtis Volin, Grahame Vittorini, and William Cody Burton. Scalable multispecies ion transport in a grid based surface-electrode trap, 2024.
 - [7] Ashley Montanaro. Quantum algorithms: an overview. *npj Quantum Information*, 2(1), January 2016. ISSN 2056-6387. doi: 10.1038/npjqi.2015.23. URL <http://dx.doi.org/10.1038/npjqi.2015.23>.
 - [8] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling Laws for Neural Language Models, 2020.
 - [9] Peter W. Shor. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM Journal on Computing*, 26(5):1484–1509, October 1997. ISSN 1095-7111. doi: 10.1137/S0097539795293172. URL <http://dx.doi.org/10.1137/S0097539795293172>.
 - [10] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing - STOC '96*, STOC '96. ACM Press, 1996. doi: 10.1145/237814.237866. URL <http://dx.doi.org/10.1145/237814.237866>.

- [11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is All you Need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- [12] James Lee-Thorp, Joshua Ainslie, Ilya Eckstein, and Santiago Ontanon. FNet: Mixing Tokens with Fourier Transforms. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4296–4313, 2022.
- [13] Andrew M. Childs and Nathan Wiebe. Hamiltonian Simulation Using Linear Combinations of Unitary Operations. *Quantum Information and Computation*, 12(11 & 12), November 2012. ISSN 1533-7146. doi: 10.26421/qic12.11-12. URL <http://dx.doi.org/10.26421/QIC12.11-12>.
- [14] András Gilyén, Yuan Su, Guang Hao Low, and Nathan Wiebe. Quantum singular value transformation and beyond: exponential improvements for quantum matrix arithmetics. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, STOC '19. ACM, June 2019. doi: 10.1145/3313276.3316366. URL <http://dx.doi.org/10.1145/3313276.3316366>.
- [15] Johannes Bausch. Recurrent Quantum Neural Networks. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1368–1379. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/0ec96be397dd6d3cf2fecb4a2d627c1c-Paper.pdf.
- [16] Dominic Widdows, Willie Aboumrar, Dohun Kim, Sayonee Ray, and Jonathan Mei. Natural language, ai, and quantum computing in 2024: Research ingredients and directions in qnlp, 2024.
- [17] Ivano Basile and Fabio Tamburini. Towards Quantum Language Models. In Martha Palmer, Rebecca Hwa, and Sebastian Riedel, editors, *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 1840–1849, Copenhagen, Denmark, September 2017. Association for Computational Linguistics. doi: 10.18653/v1/D17-1196. URL <https://aclanthology.org/D17-1196>.
- [18] El Amine Cherrat, Iordanis Kerenidis, Natansh Mathur, Jonas Landman, Martin Strahm, and Yun Yvonna Li. Quantum Vision Transformers. *Quantum*, 8:1265, February 2024. ISSN 2521-327X. doi: 10.22331/q-2024-02-22-1265. URL <http://dx.doi.org/10.22331/q-2024-02-22-1265>.
- [19] Yidong Liao and Chris Ferrie. GPT on a Quantum Computer, 2024.
- [20] Naixu Guo, Zhan Yu, Aman Agrawal, and Patrick Rebentrost. Quantum linear algebra is all you need for Transformer architectures, 2024.
- [21] Ren-Xin Zhao, Jinjing Shi, and Xuelong Li. GQHAN: A Grover-inspired Quantum Hard Attention Network, 2024.
- [22] Yeqi Gao, Zhao Song, Xin Yang, and Ruizhe Zhang. Fast quantum algorithm for attention computation, 2023.
- [23] Ren-Xin Zhao, Jinjing Shi, and Xuelong Li. QKSAN: A Quantum Kernel Self-Attention Network, 2023.
- [24] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165. PMLR, 2020. URL <http://proceedings.mlr.press/v119/katharopoulos20a.html>.
- [25] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

- [26] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, USA, 10th edition, 2011. ISBN 1107002176.
- [27] Marcello Benedetti, Erika Lloyd, Stefan Sack, and Mattia Fiorentini. Parameterized quantum circuits as machine learning models. *Quantum Science and Technology*, 4(4):043001, 2019.
- [28] John M. Martyn, Zane M. Rossi, Andrew K. Tan, and Isaac L. Chuang. Grand Unification of Quantum Algorithms. *PRX Quantum*, 2:040203, Dec 2021. doi: 10.1103/PRXQuantum.2.040203. URL <https://link.aps.org/doi/10.1103/PRXQuantum.2.040203>.
- [29] Alexander M. Dalzell, Sam McArdle, Mario Berta, Przemyslaw Bienias, Chi-Fang Chen, András Gilyén, Connor T. Hann, Michael J. Kastoryano, Emil T. Khabiboulline, Aleksander Kubica, Grant Salton, Samson Wang, and Fernando G. S. L. Brandão. Quantum algorithms: A survey of applications and end-to-end complexities, 2023.
- [30] Oscar Watts, Yuta Kikuchi, and Luuk Coopmans. Quantum Semidefinite Programming with Thermal Pure Quantum States, 2023.
- [31] Ryan Babbush, Craig Gidney, Dominic W. Berry, Nathan Wiebe, Jarrod McClean, Alexandru Paler, Austin Fowler, and Hartmut Neven. Encoding electronic spectra in quantum circuits with linear t complexity. *Physical Review X*, 8(4), October 2018. ISSN 2160-3308. doi: 10.1103/physrevx.8.041015. URL <http://dx.doi.org/10.1103/PhysRevX.8.041015>.
- [32] Mitch Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. Building a large annotated corpus of English: The Penn Treebank. *Computational linguistics*, 19(2):313–330, 1993.
- [33] Hanrui Wang, Yongshan Ding, Jiaqi Gu, Zirui Li, Yujun Lin, David Z Pan, Frederic T Chong, and Song Han. Quantumnas: Noise-adaptive search for robust quantum circuits. In *The 28th IEEE International Symposium on High-Performance Computer Architecture (HPCA-28)*, 2022.
- [34] Sukin Sim, Peter D. Johnson, and Alán Aspuru-Guzik. Expressibility and Entangling Capability of Parameterized Quantum Circuits for Hybrid Quantum-Classical Algorithms. *Advanced Quantum Technologies*, 2(12), October 2019. ISSN 2511-9044. doi: 10.1002/qute.201900070. URL <http://dx.doi.org/10.1002/qute.201900070>.
- [35] Sepp Hochreiter and Jürgen Schmidhuber. Long Short-Term Memory. *Neural Computation*, 9(8):1735–1780, 11 1997. ISSN 0899-7667. doi: 10.1162/neco.1997.9.8.1735. URL <https://doi.org/10.1162/neco.1997.9.8.1735>.
- [36] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Z. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d’Alché-Buc, Emily B. Fox, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pages 8024–8035, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>.
- [37] Dan Jurafsky and James H Martin. Speech and language processing. 3rd, 2022.
- [38] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In Yoshua Bengio and Yann LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. URL <http://arxiv.org/abs/1412.6980>.
- [39] Ilya Loshchilov and Frank Hutter. SGDR: stochastic gradient descent with warm restarts. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017. URL <https://openreview.net/forum?id=Skq89Scxx>.

- [40] Luuk Coopmans and Marcello Benedetti. On the sample complexity of quantum boltzmann machine learning. *arXiv preprint arXiv:2306.14969*, 2023.
- [41] Jarrod R. McClean, Sergio Boixo, Vadim N. Smelyanskiy, Ryan Babbush, and Hartmut Neven. Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 9 (1), November 2018. ISSN 2041-1723. doi: 10.1038/s41467-018-07090-4. URL <http://dx.doi.org/10.1038/s41467-018-07090-4>.
- [42] Gabriel Matos, Chris N. Self, Zlatko Papić, Konstantinos Meichanetzidis, and Henrik Dreyer. Characterization of variational quantum algorithms using free fermions. *Quantum*, 7:966, March 2023. ISSN 2521-327X. doi: 10.22331/q-2023-03-30-966. URL <https://doi.org/10.22331/q-2023-03-30-966>.
- [43] Amira Abbas, Robbie King, Hsin-Yuan Huang, William J. Huggins, Ramis Movassagh, Dar Gilboa, and Jarrod Ryan McClean. On quantum backpropagation, information reuse, and cheating measurement collapse. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=HF6bnhfSqH>.
- [44] M. Cerezo, Martin Larocca, Diego García-Martín, N. L. Diaz, Paolo Braccia, Enrico Fontana, Manuel S. Rudolph, Pablo Bermejo, Aroosa Ijaz, Supanut Thanasilp, Eric R. Anschuetz, and Zoë Holmes. Does provable absence of barren plateaus imply classical simulability? Or, why we need to rethink variational quantum computing, 2024.
- [45] Craig Gidney. Using quantum gates instead of ancilla bits. <https://algassert.com/circuits/2015/06/22/Using-Quantum-Gates-instead-of-Ancilla-Bits.html>, 2015. Accessed: 2024-05-21.

A Appendix

A.1 Attention in Quixer, $d > 2$

As mentioned in Section 3.4.1, higher degree polynomials in the QSVT compute interactions between skip- k -grams. Indeed, from Eq. (3) and Eq. (14),

$$P(M_{\vec{b},\theta}) = P\left(\sum_j^{n-1} b_j U_j\right) \quad (35)$$

$$= \sum_{k=0}^d c_k \left(\sum_{j=0}^{n-1} b_j U_j\right)^k \quad (36)$$

$$= \sum_{k=1}^d c_k \left[\sum_{\vec{\alpha} \in \{1, \dots, n-1\}^k} b_{\alpha_1} \dots b_{\alpha_k} U_{\alpha_1} \dots U_{\alpha_k} \right] + c_0 I. \quad (37)$$

This is the sum of the composition of unitaries associated with all skip- k -grams, for $k \in \{1 \dots d\}$.

A.2 Auxiliary result

The following lemma proves that the LCU circuit defined in Eq. (7) yields the desired result.

Lemma 1. *For the circuit defined in Eq. (7), it is the case that*

$$(\langle 0| \otimes I) U_M (|0\rangle \otimes I) = \sum_{i=0}^{n-1} |a_i|^2 U_i. \quad (38)$$

Proof.

$$M := (\langle 0| \otimes I) U_M (|0\rangle \otimes I) \quad (39)$$

$$= (\langle 0| U_{\text{PREP}}^\dagger \otimes I) U_{\text{SEL}} (U_{\text{PREP}} |0\rangle \otimes I) \quad (40)$$

$$= (\langle a| \otimes I) U_{\text{SEL}} (|a\rangle \otimes I) \quad (41)$$

$$= \left(\sum_{i=0}^{n-1} \bar{a}_i \langle i| \otimes I \right) \sum_{j=0}^{n-1} |j\rangle \langle j| \otimes U_j \left(\sum_{k=0}^{n-1} a_k |k\rangle \otimes I \right) \quad (42)$$

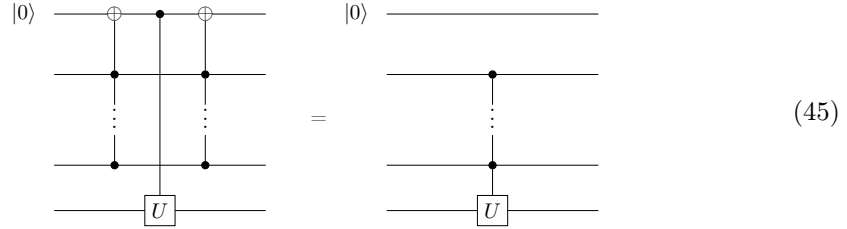
$$= \sum_{i,j,k=0}^{n-1} \delta_{ij} \delta_{jk} a_i \bar{a}_k U_j \quad (43)$$

$$= \sum_i |a_i|^2 U_i \quad (44)$$

□

A.3 Toffoli construction

Gidney [45] describes a method to construct a Toffoli gate [26] controlled on n qubits using a number of gates which is linear in n . If U is a single-qubit unitary, by conjugating it by these Toffoli gates as is done in (45), one can implement a version of that unitary controlled on n qubits in a linear number of gates.



$$(45)$$

A.4 Hyperparameters

Table 2 lists the seed and hyperparameter used for each of our runs, as used by the source code.

Model	Dimension	Seed	Model	Dimension	Seed
FNet	128	865026	Transformer	128	753912
FNet	128	896680	Transformer	128	740072
FNet	128	157158	Transformer	128	73815
FNet	128	912324	Transformer	128	233689
FNet	128	336635	Transformer	128	791410
FNet	128	47752	Transformer	128	845853
FNet	128	521512	Transformer	128	256815
FNet	128	410043	Transformer	128	616390
FNet	128	906901	Transformer	128	494215
FNet	128	528631	Transformer	128	664970
FNet	96	544610	Transformer	96	520839
FNet	96	435960	Transformer	96	799108
FNet	96	855886	Transformer	96	102749
FNet	96	255092	Transformer	96	143407
FNet	96	325162	Transformer	96	687589
FNet	96	44219	Transformer	96	418518
FNet	96	613756	Transformer	96	861454
FNet	96	969277	Transformer	96	967387
FNet	96	977070	Transformer	96	725501
FNet	96	27859	Transformer	96	178408
LSTM	128	110433	Quixer	6 qbs	363500
LSTM	128	581765	Quixer	6 qbs	844031
LSTM	128	78554	Quixer	6 qbs	707858
LSTM	128	776951	Quixer	6 qbs	734571
LSTM	128	312284	Quixer	6 qbs	134017
LSTM	128	586743	Quixer	6 qbs	246154
LSTM	128	125848	Quixer	6 qbs	631481
LSTM	128	956941	Quixer	6 qbs	168344
LSTM	128	80607	Quixer	6 qbs	356044
LSTM	128	889548	Quixer	6 qbs	682772
LSTM	96	124900			
LSTM	96	951780			
LSTM	96	927932			
LSTM	96	217285			
LSTM	96	627139			
LSTM	96	694772			
LSTM	96	311736			
LSTM	96	939997			
LSTM	96	687992			
LSTM	96	446349			

Table 2: Seeds used for each run for each model, randomly generated using `torch.randint`.